

AI VIET NAM

@aivietnam.edu.vn

Vector and Matrix for High-level Notations

Quang-Vinh Dinh
PhD in Computer Science

Objectives

Vector and Matrix

$$\begin{array}{|c|c|} \hline \text{X} & \\ \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} \cdot \begin{array}{|c|} \hline \text{v} \\ \hline 1 \\ \hline 2 \\ \hline \end{array} = \begin{array}{|c|} \hline \text{result} \\ \hline 5 \\ \hline 11 \\ \hline \end{array}$$

$$\begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} \xrightarrow{\text{T}} \begin{array}{|c|c|} \hline 1 & 3 \\ \hline 2 & 4 \\ \hline \end{array}$$

Background Subtraction



Notations

$$ij \rightarrow ji$$

$$\begin{array}{c} j \downarrow \\ \begin{array}{|c|c|c|} \hline 0 & 2 & 1 \\ \hline 3 & 1 & 4 \\ \hline \end{array} \\ A \end{array} \quad \begin{array}{c} k \downarrow \\ j \rightarrow \begin{array}{|c|c|} \hline 2 & 5 \\ \hline 0 & 1 \\ \hline 3 & 4 \\ \hline \end{array} \\ A^T \end{array}$$

Outline

SECTION 1

Vector and Matrix

$$\begin{matrix} & \mathbf{x} \\ \begin{matrix} 1 & 2 \\ 3 & 4 \end{matrix} & \cdot \end{matrix} \begin{matrix} \mathbf{v} \\ \begin{matrix} 1 \\ 2 \end{matrix} \end{matrix} = \begin{matrix} \mathbf{result} \\ \begin{matrix} 5 \\ 11 \end{matrix} \end{matrix}$$

SECTION 2

Background Subtraction

SECTION 3

Notations

$$\begin{matrix} \begin{matrix} 1 & 2 \\ 3 & 4 \end{matrix} \end{matrix}^{\mathbf{T}} \rightarrow \begin{matrix} \begin{matrix} 1 & 3 \\ 2 & 4 \end{matrix} \end{matrix}$$

Introduction

House price data

Feature		Label	
	area	price	
	6.7	9.1	
	4.6	5.9	
	3.5	4.6	
	5.5	6.7	

$$\text{price} = a * \text{area} + b$$

Features			Label
TV	↕ Radio	↕ Newspaper	↕ Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

$$\text{Model: } \hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$$

$$\text{Sale} = w_1 * TV + w_2 * Radio + w_3 * Newspaper + b$$

Boston House
Price Data

Features													Label
crim	↕ zn	↕ indus	↕ chas	↕ nox	↕ rm	↕ age	↕ dis	↕ rad	↕ tax	↕ ptratio	↕ black	↕ lstat	↕ medv
0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9

$$\text{medv} = w_1 * x_1 + \dots + w_{13} * x_{13} + b$$

❖ System of linear equations

Features			Label
TV	↕ Radio	↕ Newspaper	↕ Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12

$Sale = a_1 * TV + a_2 * Radio + a_3 * Newspaper + b$

$a_1T_1 + a_2R_1 + a_3N_1 = S_1$
 $a_1T_2 + a_2R_2 + a_3N_2 = S_2$
 $a_1T_3 + a_2R_3 + a_3N_3 = S_3$

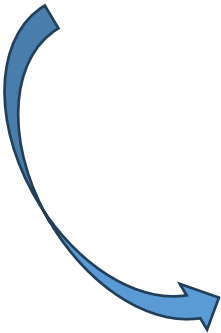
Set b=0 for the sake of simplicity

$$a_1 \begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix} + a_2 \begin{bmatrix} R_1 \\ R_2 \\ R_3 \end{bmatrix} + a_3 \begin{bmatrix} N_1 \\ N_2 \\ N_3 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \\ S_3 \end{bmatrix}$$



$$\begin{bmatrix} T_1 & R_1 & N_1 \\ T_2 & R_2 & N_2 \\ T_3 & R_3 & N_3 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} S_1 \\ S_2 \\ S_3 \end{bmatrix}$$

A x b



$Ax = b$



❖ Introduction

Vector

n is a natural number

$\mathcal{R}$ is a set of real numbers

$\vec{v}$ has a length of n and contain real numbers

$$\vec{v} \in \mathcal{R}^n$$

$$\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} \in \begin{bmatrix} \mathcal{R} \\ \mathcal{R} \\ \mathcal{R} \end{bmatrix} = \mathcal{R}^3$$

Matrix

Matrix A has the shape of rectangle

Has m rows and n columns

Use capital letter

$$A \in \mathcal{R}^{m \times n}$$

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \in \begin{bmatrix} \mathcal{R} & \mathcal{R} \\ \mathcal{R} & \mathcal{R} \\ \mathcal{R} & \mathcal{R} \end{bmatrix} = \mathcal{R}^{3 \times 2}$$

Vector Addition

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_3 \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_3 \end{bmatrix}$$

$$\vec{v} + \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_3 \end{bmatrix} + \begin{bmatrix} u_1 \\ \dots \\ u_3 \end{bmatrix} = \begin{bmatrix} v_1 + u_1 \\ \dots \\ v_3 + u_3 \end{bmatrix}$$

data x		data y		result
1		5		6
2		6		8
3	+	7	=	10
4		8		12

```
1 def add_vectors(vector1, vector2):
2     '''
3     Add corresponding elements between two vectors
4     vector1 and vector2 are with list type
5     output is a vector (list)
6     '''
7
8     return [v1+v2 for v1, v2 in zip(vector1, vector2)]
```

```
2 import numpy as np
3
4 x = np.array([1,2,3,4])
5 y = np.array([5,6,7,8])
6
7 print('data x \n', x)
8 print('data y \n', y)
9
10 # Tổng của 2 mảng
11 print('method 1 \n', x + y)
12 print('method 2 \n', np.add(x, y))
```

```
data x
[1 2 3 4]
data y
[5 6 7 8]
method 1
[ 6  8 10 12]
method 2
[ 6  8 10 12]
```

Vector Operations

❖ Vector subtraction

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} - \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} - \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1 - u_1 \\ \dots \\ v_n - u_n \end{bmatrix}$$

```
2 import numpy as np
3
4 x = np.array([5,6,7,8])
5 y = np.array([1,2,3,4])
6
7 print('data x \n', x)
8 print('data y \n', y)
9
10 # Hiệu 2 mảng
11 print('method 1 \n', x - y)
12 print('method 2 \n', np.subtract(x, y))
```

```
data x
[5 6 7 8]
data y
[1 2 3 4]
method 1
[4 4 4 4]
method 2
[4 4 4 4]
```

data x		data y		result
5		1		4
6		2		4
7	-	3	=	4
8		4		4



Matrix Operations

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1n} \\ \dots & \dots & \dots \\ b_{m1} & \dots & b_{mn} \end{bmatrix}$$

Addition

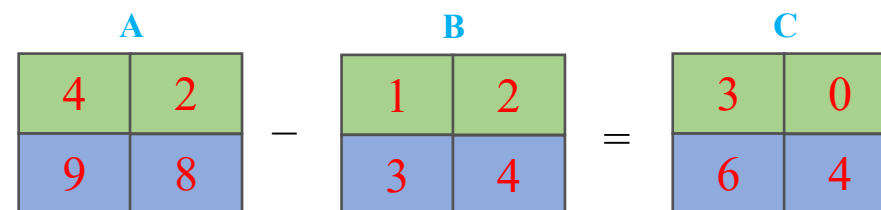
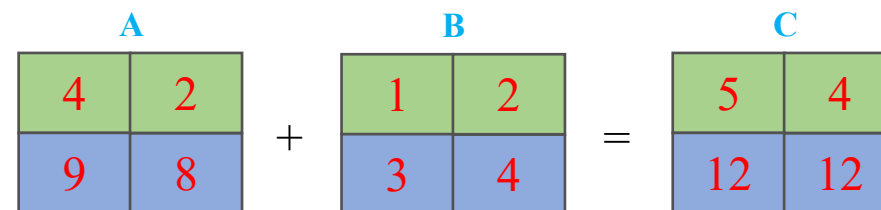
$$A + B = \begin{bmatrix} (a_{11} + b_{11}) & \dots & (a_{1n} + b_{1n}) \\ \dots & \dots & \dots \\ (a_{m1} + b_{m1}) & \dots & (a_{mn} + b_{mn}) \end{bmatrix}$$

Subtraction

$$A - B = \begin{bmatrix} (a_{11} - b_{11}) & \dots & (a_{1n} - b_{1n}) \\ \dots & \dots & \dots \\ (a_{m1} - b_{m1}) & \dots & (a_{mn} - b_{mn}) \end{bmatrix}$$

```
1 import numpy as np
2
3 A = np.array([[4,2],
4               [9,8]])
5 B = np.array([[1,2],
6               [3,4]])
7
8 print(A+B)
9 print(A-B)
```

```
[[ 5  4]
 [12 12]]
[[ 3  0]
 [ 6  4]]
```



Hadamard product

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} \odot \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \odot \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1 \times u_1 \\ \dots \\ v_n \times u_n \end{bmatrix}$$

$$u \odot v \rightarrow z = \{z_0, z_1, \dots, z_n\}$$
$$z_i = u_i v_i$$

data x		data y		result
1	$\odot$	5	=	5
2		6		12
3		7		21
4		8		32

```
1 def Hadamard_product(vector1, vector2):  
2     '''  
3     Compute Hadamard product between two vectors  
4     Output is a vector  
5     '''  
6     return [v1*v2 for v1, v2 in zip(vector1, vector2)]
```

```
2 import numpy as np  
3  
4 x = np.array([1,2,3,4])  
5 y = np.array([5,6,7,8])  
6  
7 print('data x \n', x)  
8 print('data y \n', y)  
9  
10 # Tích các phần tử tương ứng giữa x và y  
11 print('method 1 \n', x*y)  
12 print('method 2 \n', np.multiply(x, y))
```

```
data x  
[1 2 3 4]  
data y  
[5 6 7 8]  
method 1  
[ 5 12 21 32]  
method 2  
[ 5 12 21 32]
```

❖ Hadamard Division

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} \oslash \vec{u} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \oslash \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} v_1/u_1 \\ \dots \\ v_n/u_n \end{bmatrix}$$

data x		data y		result
1		5		0.2
2		6		0.33
3	$\oslash$	7	=	0.42
4		8		0.5

```
2 import numpy as np
3
4 x = np.array([1,2,3,4])
5 y = np.array([5,6,7,8])
6
7 print('data x \n', x)
8 print('data y \n', y)
9
10 # Phép chia các từng phần tương ứng x cho y
11 print('method 1 \n', x / y)
12 print('method 2 \n', x // y)
13 print('method 3 \n', np.divide(x, y))
```

```
data x
[1 2 3 4]
data y
[5 6 7 8]
method 1
[0.2          0.33333333 0.42857143 0.5          ]
method 2
[0 0 0 0]
method 3
[0.2          0.33333333 0.42857143 0.5          ]
```

Matrix Operations

Matrix-vector multiplication

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad \vec{x} = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}$$

$$A \in \mathcal{R}^{m \times n}$$

$$\mathbf{c} = A\vec{x} \quad \text{where } c_i = \sum_{l=1}^n a_{il}x_l$$

X			v		result
1	2	•	1	=	5
3	4		2		11

```
1 # aivietnam.ai
2 import numpy as np
3
4 X = np.array([[1,2],
5               [3,4]])
6 v = np.array([1,2])
7
8 print('matrix X \n', X)
9 print('vector v \n', v)
10
11 # phép nhân giữa ma trận và vector
12 print('method 1: X.dot(v) \n', X.dot(v))
13 print('method 1: v.dot(X) \n', v.dot(X))
14 #print('\n method 2: X.dot(v) \n', np.dot(X, v))
15 #print('\n method 2: v.dot(X) \n', np.dot(v, X))
```

```
matrix X
[[1 2]
 [3 4]]
vector v
[1 2]
method 1: X.dot(v)
[ 5 11]
method 1: v.dot(X)
[ 7 10]
```

			X		result	
v	1	2	•	=	7	10
	3	4				

Vector Operations

Multiply with a number

$$\alpha \vec{u} = \alpha \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} \alpha u_1 \\ \dots \\ \alpha u_n \end{bmatrix}$$

$$\alpha \mathbf{u} \rightarrow \mathbf{z} = \{z_0, z_1, \dots, z_n\}$$

$$z_i = \alpha u_i$$

Length of a vector

$$\|\vec{u}\| = \sqrt{u_1^2 + \dots + u_n^2}$$

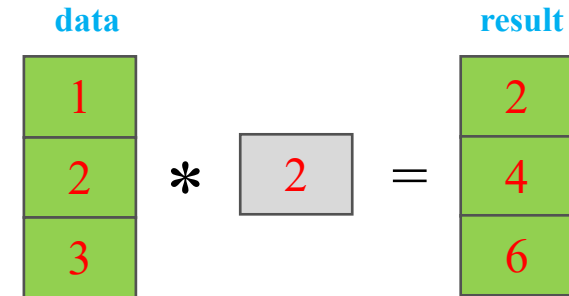
Squared norm $\|\mathbf{u}\|^2$

$$\vec{u} = \begin{bmatrix} 1 & 2 & 4 & 2 \end{bmatrix}$$

$$\|\vec{u}\| = \sqrt{1^2 + 2^2 + 4^2 + 2^2} = 5$$

```
2 import numpy as np
3
4 # create data
5 data = np.array([1,2,3])
6 factor = 2
7
8 # broadcasting
9 result_multiplication = data*factor
```

```
[1 2 3]
[2 4 6]
```



```
1 # compute length of a vector
2
3 import numpy as np
4
5 data = np.array([1, 2, 4, 2])
6 length = np.linalg.norm(data)
7 print(length)
```

5.0

Vector Operations

❖ Dot product

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \quad \vec{u} = \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix}$$

$$\vec{v} \cdot \vec{u} = v_1 \times u_1 + \dots + v_n \times u_n$$

$$\langle x, y \rangle = x^T y = \sum_i x_i y_i$$

v			w		result
1	2	•	2	=	8
			3		

```
1 def dot_product(vector1, vector2):
2     '''
3     Compute dot product between two vectors
4     Output is a floating-point number
5     '''
6
7     return sum([v1*v2 for v1, v2 in zip(vector1, vector2)])
8
9 # test case
10 vector1 = [1, 2, 3]
11 vector2 = [2, 3, 4]
12
13 ouptut = dot_product(vector1, vector2)
14 print(ouptut)
```

20

```
2 import numpy as np
3
4 v = np.array([1, 2])
5 w = np.array([2, 3])
6
7 # Tính inner product giữa v và w
8 print('method 1 \n', v.dot(w))
9 print('method 2 \n', np.dot(v, w))
```

method 1
8

method 2
8

❖ Dot product

Measure angles between vectors and computes the length of a vector

Definition. The *dot product* of two vectors x, y in $\mathbf{R}^n$ is

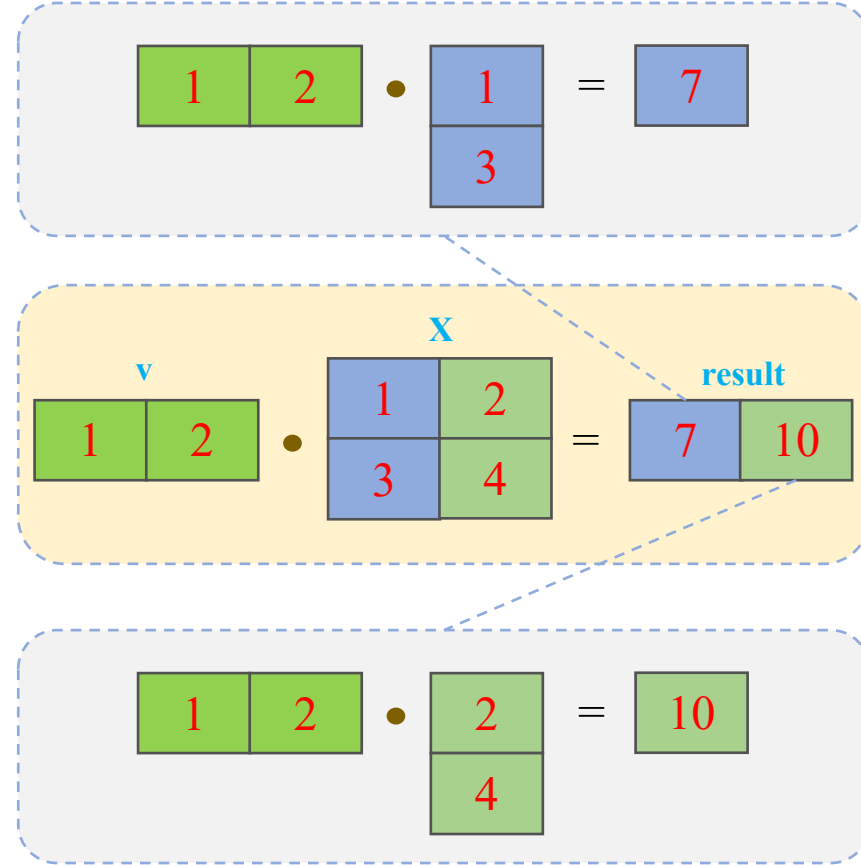
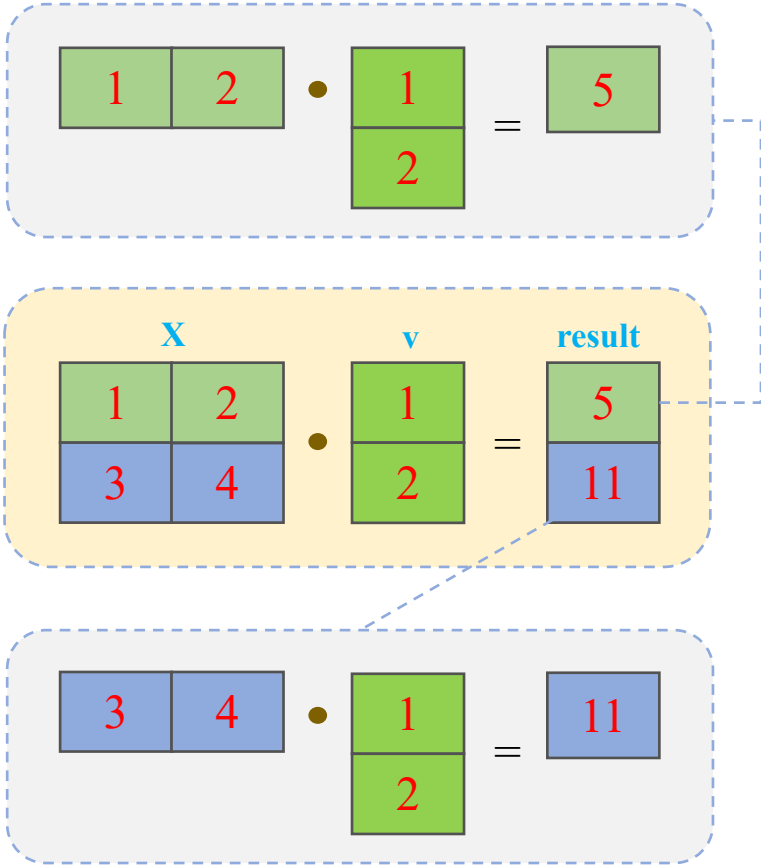
$$x \cdot y = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \cdot \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n$$

Thinking of x, y as column vectors, this is the same as $x^T y$.

The *length* of a vectors x in $\mathbf{R}^n$ is the number

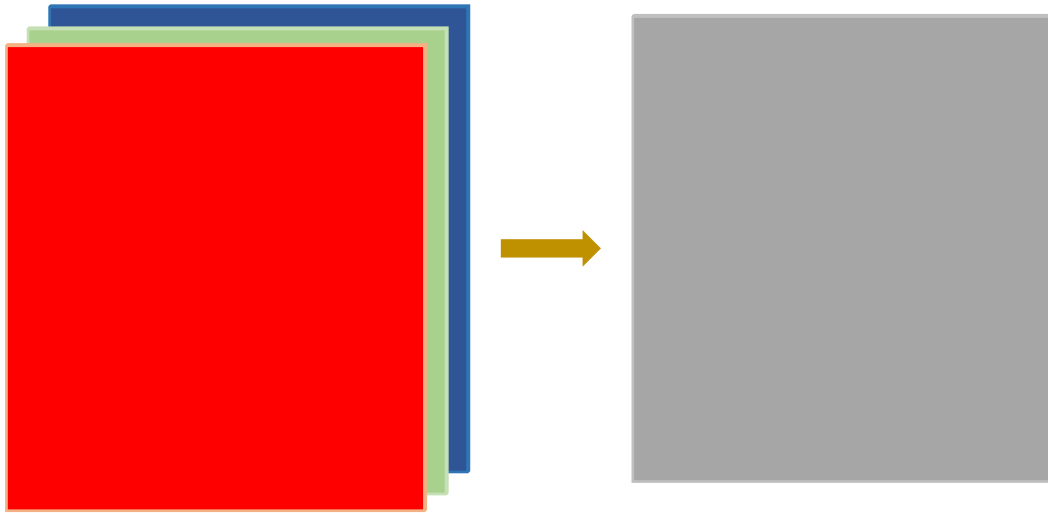
$$\|x\| = \sqrt{x \cdot x} = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

❖ Dot product



Vector Operations

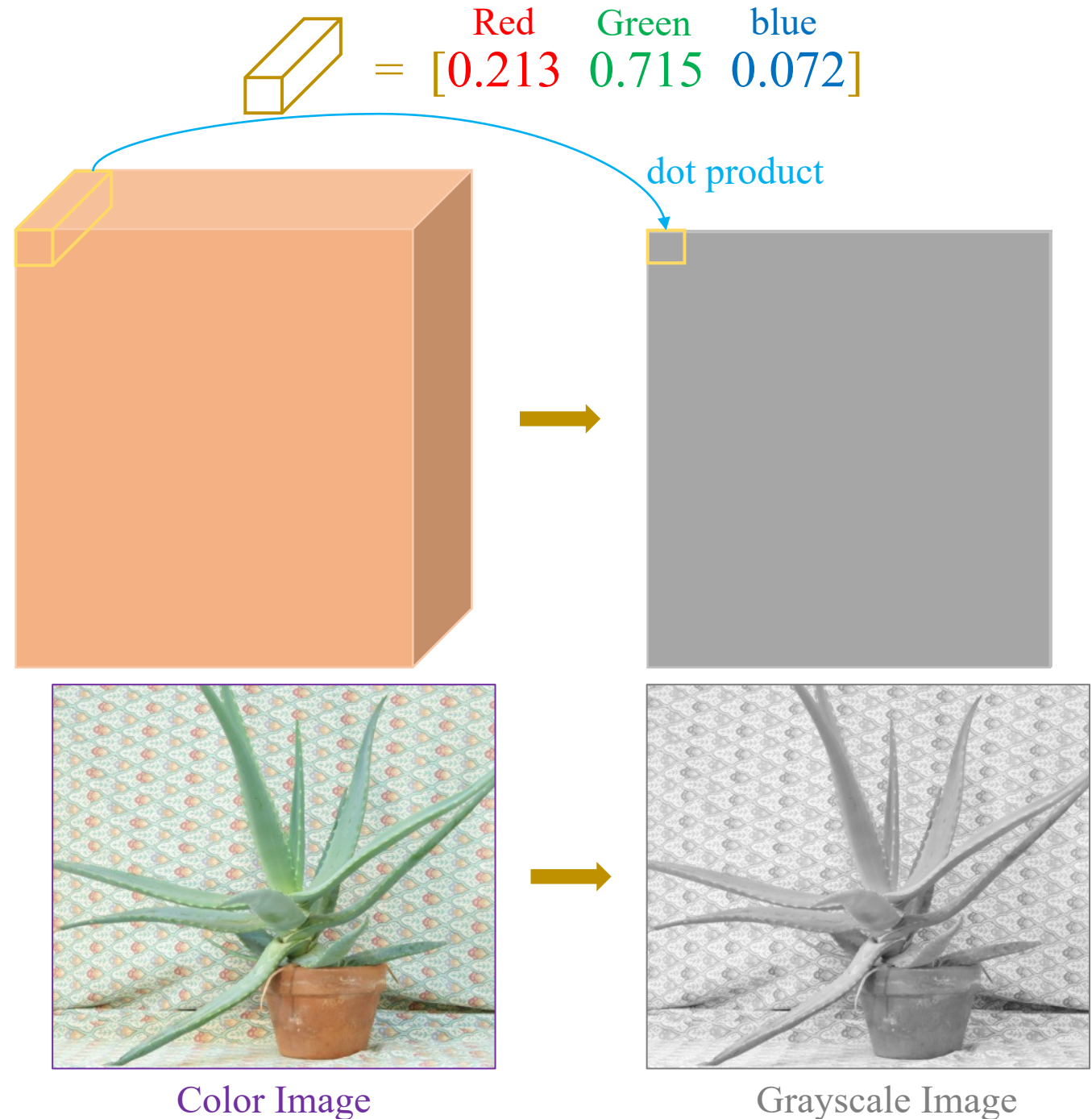
Color → Grayscale



3-channel Image
(Red, Green, blue)

1-channel Image

```
1 # load color image (channel order: BGR)
2 color_image = cv2.imread('aloe.png')
3
4 # conversion vector [B, G, R]
5 vector = np.array([0.072, 0.715, 0.213])
6
7 # convert
8 grayscale_image = color_image @ vector
```



Color Image

Grayscale Image

❖ Matrix-matrix multiplication

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1d} \\ \dots & \dots & \dots \\ b_{n1} & \dots & b_{nd} \end{bmatrix}$$
$$A \in \mathcal{R}^{m \times n} \quad B \in \mathcal{R}^{n \times d}$$

$$C = AB \in \mathcal{R}^{m \times d}$$
$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

$$AB = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} b_{11} & \dots & b_{1k} \\ \dots & \dots & \dots \\ b_{n1} & \dots & b_{nk} \end{bmatrix} = \begin{bmatrix} (a_{11}b_{11} + \dots + a_{1n}b_{n1}) & \dots & (a_{11}b_{1k} + \dots + a_{1n}b_{nk}) \\ \dots & \dots & \dots \\ (a_{m1}b_{11} + \dots + a_{mn}b_{n1}) & \dots & (a_{m1}b_{1k} + \dots + a_{mn}b_{nk}) \end{bmatrix}$$

Matrix Operations

❖ Matrix-matrix multiplication

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1d} \\ \dots & \dots & \dots \\ b_{n1} & \dots & b_{nd} \end{bmatrix}$$

$$A \in \mathcal{R}^{m \times n}$$

$$B \in \mathcal{R}^{n \times d}$$

$$C = AB \in \mathcal{R}^{m \times d}$$

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

```
1 def matrix_multiplication(matrix1, matrix2):
2     '''
3     This function does the multiplication between two matrices.
4     #columns of matrix1 == #rows of matrix2
5     '''
6     matrix1_nrows = len(matrix1)
7     matrix1_ncols = len(matrix1[0])
8
9     matrix2_nrows = len(matrix2)
10    matrix2_ncols = len(matrix2[0])
11
12    # tạo matrix kết quả
13    result = [[0]*matrix2_ncols for i in range(matrix1_nrows)]
14
15    for i in range(matrix1_nrows):
16        for j in range(matrix2_ncols):
17            for k in range(matrix2_nrows):
18                result[i][j] += matrix1[i][k] * matrix2[k][j]
19
20    return result
21
22    # test case
23    # 3x3 matrix
24    matrix1 = [[1, 2, 3],
25               [4, 5, 6],
26               [7, 8, 9]]
27
28    # 3x4 matrix
29    matrix2 = [[1, 1, 2, 1],
30               [1, 2, 1, 1],
31               [1, 1, 1, 2]]
32
33    result = matrix_multiplication(matrix1, matrix2)
34    print(result[0])
35    print(result[1])
36    print(result[2])
```

```
[6, 8, 7, 9]
[15, 20, 19, 21]
[24, 32, 31, 33]
```

Matrix-matrix multiplication

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & \dots & b_{1d} \\ \dots & \dots & \dots \\ b_{n1} & \dots & b_{nd} \end{bmatrix}$$

$$A \in \mathcal{R}^{m \times n}$$

$$B \in \mathcal{R}^{n \times d}$$

$$C = AB \in \mathcal{R}^{m \times d}$$

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

X			Y			result	
1	2	•	2	3	=	6	5
3	4		2	1		14	13

```
1 # aivietnam.ai
2 import numpy as np
3
4 X = np.array([[1,2],
5               [3,4]])
6 Y = np.array([[2,3],
7               [2,1]])
8
9 # Phép nhân giữa hai ma trận
10 print('method 1 \n', X.dot(Y))
11 print('method 1 \n', Y.dot(X))
12 #print('method 2 \n', np.dot(X, Y))
13 #print('method 2 \n', np.dot(Y, X))
```

method 1

```
[[ 6  5]
 [14 13]]
```

method 1

```
[[11 16]
 [ 5  8]]
```

Y			X			result	
2	3	•	1	2	=	11	16
2	1		3	4		5	8

Matrix Operations

Transpose

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix}$$

$$A^T = \begin{bmatrix} a_{11} & \dots & a_{m1} \\ \dots & \dots & \dots \\ a_{1n} & \dots & a_{mn} \end{bmatrix}$$

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix}$$

$$\vec{v}^T = [v_1 \ \dots \ v_n]$$

Example

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}$$

$$A^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \end{bmatrix}$$

```

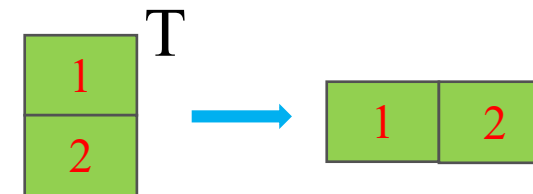
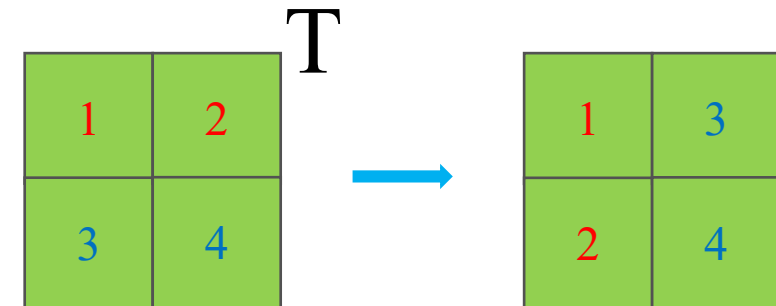
2 import numpy as np
3
4 X = np.array([[1,2],
5               [3,4]])
6 print(X)
7
8 #chuyển vị
9 print(X.T)

```

```

[[1 2]
 [3 4]]
[[1 3]
 [2 4]]

```



❖ Other Hadamard operations

data		result
1	sqrt(data) =	1.0
2		1.4
3		1.7
4		2.0

$$\mathbf{X} \sim (n, d)$$

$$\mathbf{Z} = \exp(\mathbf{X}) \sim (n, d)$$

$$Z_{ij} = [\exp(\mathbf{X})]_{ij} = \exp(X_{ij})$$

```
1 # aivietnam.ai
2 import numpy as np
3
4 data = np.array([1, 2, 3, 4])
5
6 print('data \n', data)
7
8 # Căn bậc 2 từng phần tử trong data
9 print('sqrt \n', np.sqrt(data))
```

```
data
[1 2 3 4]
sqrt
[1.          1.41421356  1.73205081  2.          ]
```

Outline

SECTION 1

Vector and Matrix

SECTION 2

Background Subtraction

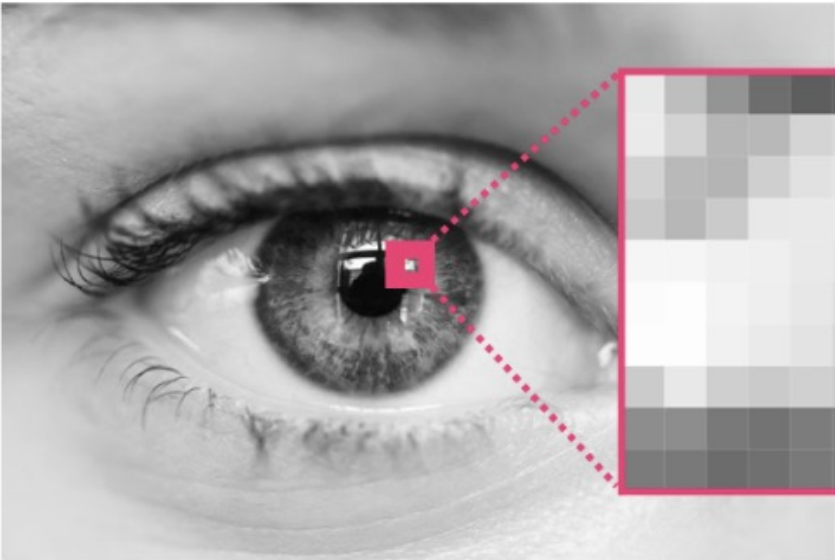
SECTION 3

Notations



❖ Grayscale images

Height



Width

230	194	147	108	90	98	84	96	91	101
237	206	188	195	207	213	163	123	116	128
210	183	180	205	224	234	188	122	134	147
198	189	201	227	229	232	200	125	127	135
249	241	237	244	232	226	202	116	125	126
251	254	241	239	230	217	196	102	103	99
243	255	240	231	227	214	203	116	95	91
204	231	208	200	207	201	200	121	95	95
144	140	120	115	125	127	143	118	92	91
121	121	108	109	122	121	134	106	86	97

Pixel p = scalar

$0 \leq p \leq 255$

Resolution: #pixels

Resolution = Height×Width

❖ Color images



$$\text{Pixel } p = \begin{bmatrix} r \\ g \\ b \end{bmatrix}$$

$$0 \leq r, g, b \leq 255$$



Resolution: #pixels
Resolution = Height $\times$ Width

Background Subtraction

❖ Some important functions

where() function

arr =	0	1	2	3	4
arr<3 =	T	T	T	F	F
out =	0	1	2	6	8

```
4 # create an array
5 arr = np.arange(5)
6 print(arr)
7
8 # condition
9 condition = arr < 3
10 out = np.where(condition, arr, arr*2)
11
12 print(condition)
13 print(out)
```

```
[0 1 2 3 4]
[ True  True  True False False]
[0 1 2 6 8]
```

```
_, output = cv2.threshold(input,
                           threshold, max_value,
                           cv2.THRESH_BINARY)
```

```
1 import numpy as np
2 import cv2
3
4 data = np.array([[0, 63, 174],
5                  [30, 205, 132],
6                  [52, 178, 210]])
7 data = data.reshape(3, 3)
8 data = data.astype(np.uint8)
9 print(data)
10
11 _, output = cv2.threshold(data, 100, 255,
12                             cv2.THRESH_BINARY)
13 print(output)
```

```
[[ 0  63 174]
 [ 30 205 132]
 [ 52 178 210]]
[[ 0  0 255]
 [ 0 255 255]
 [ 0 255 255]]
```

Background Subtraction

How?

Background **B**



FakeBackground **F**



Coming image **I**



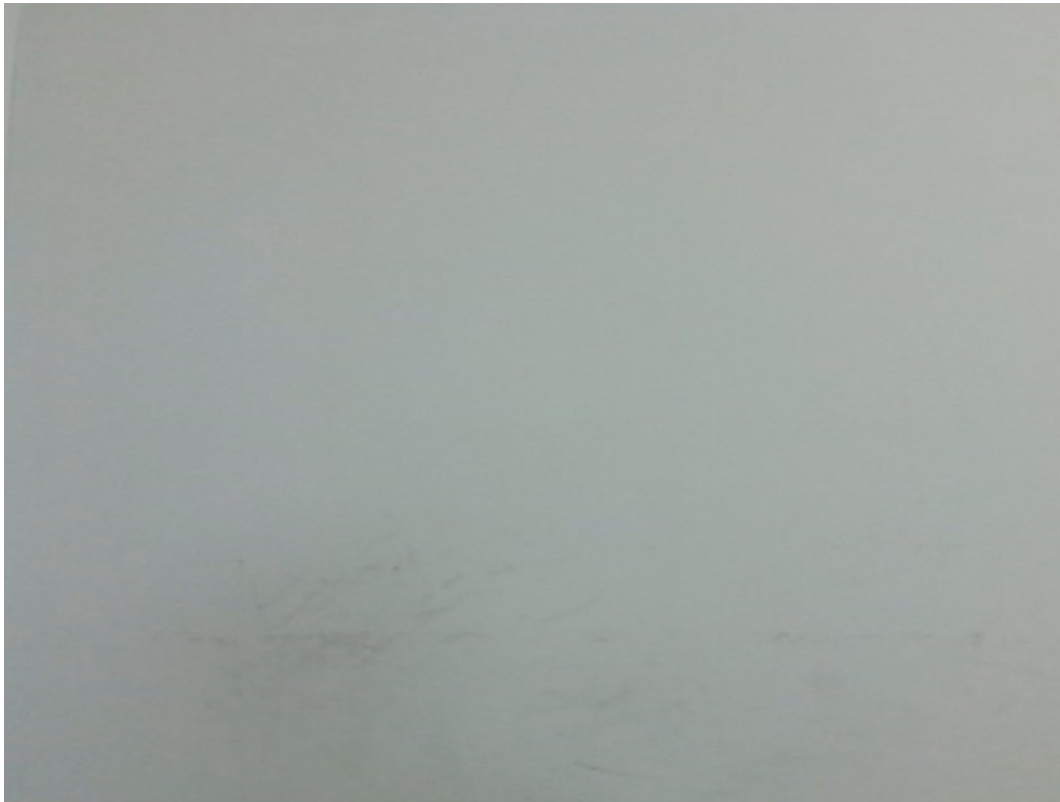
Output **O**



Background Subtraction

❖ Extract background/foreground

Background



coming image



Background B



Coming image I



Background Subtraction

$$A = |I - B|$$

$$M = \begin{cases} 0 & \text{if } A < th \\ 1 \text{ (or 255) } & \text{otherwise} \end{cases}$$

```
import cv2
```

```
bg = cv2.imread('background.png', 1)
```

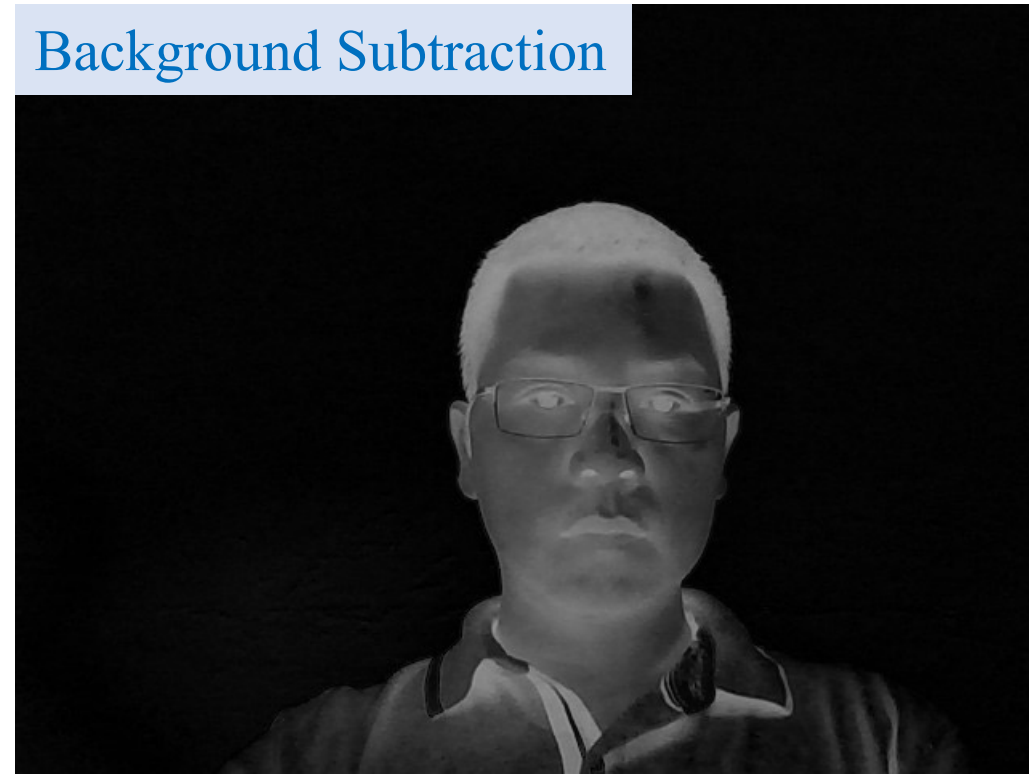
```
bg = cv2.resize(bg, (640, 480))
```

```
img = cv2.imread('StillImage.png', 1)
```

```
img = cv2.resize(img, (640, 480))
```

```
difference = cv2.absdiff(bg, img)
```

Background Subtraction



Background Subtraction

```
import cv2

bg = cv2.imread('background.png', 1)
bg = cv2.resize(bg, (640, 480))

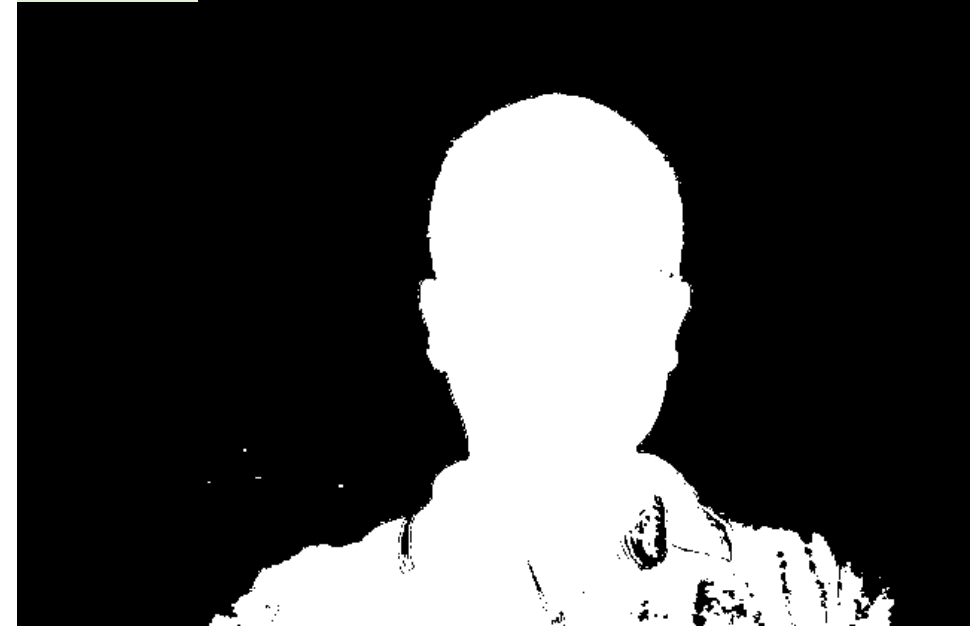
img = cv2.imread('StillImage.png', 1)
img = cv2.resize(img, (640, 480))

difference = cv2.absdiff(bg, img)
_, difference_binary = cv2.threshold(difference, 15, 255,
                                     cv2.THRESH_BINARY)
```

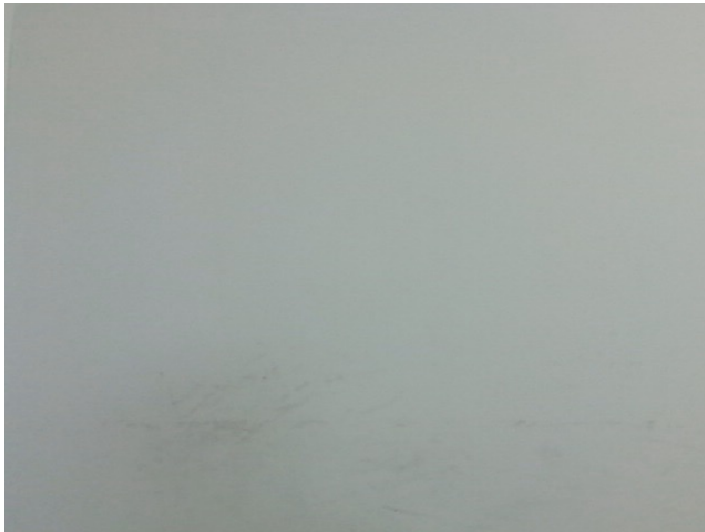
Subtracted Background



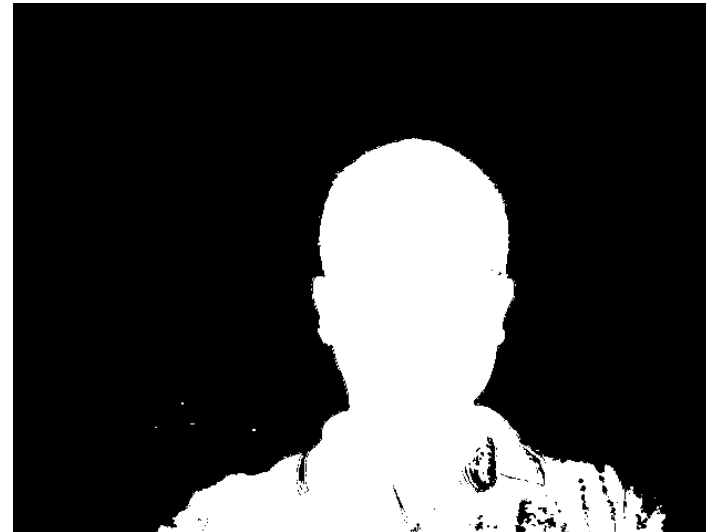
Mask



Background



Mask



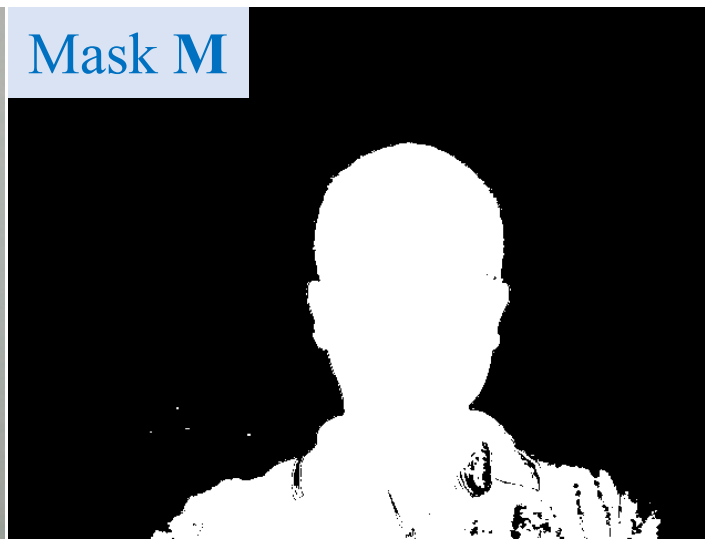
Coming image



FakeBackground



Background Subtraction

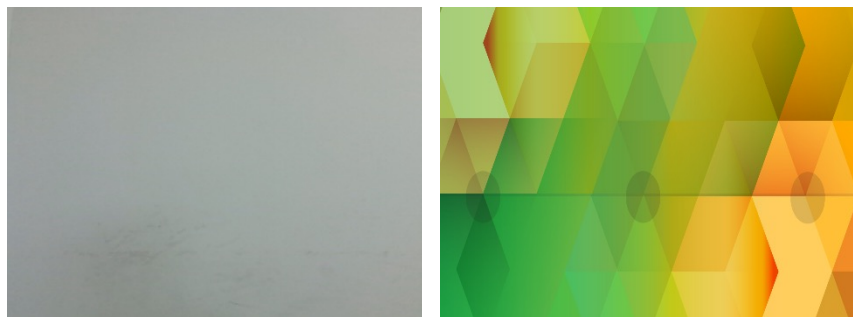


$$A = |I - B|$$

$$M = \begin{cases} 0 & \text{if } A < th \\ 1 & \text{otherwise} \end{cases}$$

$$O = \begin{cases} F & \text{if } M == 0 \\ I & \text{otherwise} \end{cases}$$

$th = 15$



+



```
1 import numpy as np
2 import cv2
3
4 bg = cv2.imread('background.png', 1)
5 bg = cv2.resize(bg, (640, 480))
6
7 img = cv2.imread('StillImage.png', 1)
8 img = cv2.resize(img, (640, 480))
9
10 difference = cv2.absdiff(bg, img)
11 _, difference_binary = cv2.threshold(difference, 15, 255,
12                                     cv2.THRESH_BINARY)
13
14 new_bg = cv2.imread('FakeBackground.png')
15 new_bg = cv2.resize(new_bg, (640, 480))
16
17 output = np.where(difference_binary==0, new_bg, img)
18 cv2.imwrite('output_111.png', output)
```



Background Subtraction

The screenshot shows a Jupyter Notebook titled "OpenCV" running on a local host. The notebook contains a Python script for background subtraction. The script starts by reading the first frame from a video capture object, applying Gaussian blur, and displaying it. It then enters a while loop to capture frame-by-frame, applying Gaussian blur to each frame. The difference between the first frame and the current frame is calculated using cv2.absdiff, summed along the second axis, and thresholded using cv2.threshold. The resulting difference image is then stacked with the original frame to create a 3D volume. The script also displays the original frame, the current frame, the difference image, and the output of the thresholding operation. A key press (27) is used to break the loop, and the video capture object is released.

```
14
15 _, first_frame = cap.read()
16 first_frame = cv2.GaussianBlur(first_frame, (5, 5), 0)
17 cv2.imshow('Window-Title', first_frame)
18 cv2.waitKey(0)
19
20 # get live video
21 while(True):
22     # Capture frame-by-frame
23     _, frame = cap.read()
24     frame = cv2.GaussianBlur(frame, (5, 5), 0)
25
26     difference = cv2.absdiff(first_frame, frame)
27     difference = np.sum(difference, axis=2) / 3.0
28     _, difference = cv2.threshold(difference, 25, 255, cv2.THRESH_BINARY)
29
30     diff3 = np.stack([difference, difference, difference], axis=2)
31     output = np.where(diff3==0, bg, frame)
32
33     cv2.imshow("First frame", first_frame)
34     cv2.imshow("Frame", frame)
35     cv2.imshow("difference", difference)
36     cv2.imshow("output", output)
37
38     key = cv2.waitKey(30)
39     if key == 27:
40         break
41
42 cv2.waitKey(0)
43 cv2.destroyAllWindows()
44 cap.release()
```

In [8]: 1 import numpy as np

Outline

SECTION 1

Vector and Matrix

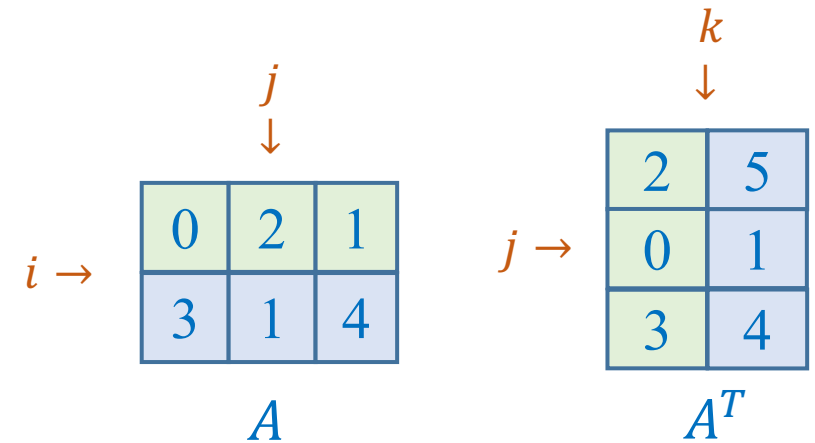
SECTION 2

Background Subtraction

SECTION 3

Notations

$$ij \rightarrow ji$$



Notations

❖ argmin and argmax

	i, j			
	0	1	2	3
$\mathbf{x} =$	4	7	1	5
	x_0	x_1	x_2	x_3
	x_i, x_j			

argmin (argument of the minimum)

$$\underset{i}{\operatorname{argmin}}(\mathbf{x}) = \{i \in D \mid x_i \leq x_j \text{ for all } j \in D\}$$

argmax (argument of the maximum)

$$\underset{i}{\operatorname{argmax}}(\mathbf{x}) = \{i \in D \mid x_i \geq x_j \text{ for all } j \in D\}$$

```
x = np.array([4, 7, 1, 5])
result = np.argmin(x)
print(result)
```

✓ 0.0s

2

	0	1	2	3
$\mathbf{x} =$	4	7	1	5

```
x = np.array([4, 7, 1, 5])
result = np.argmax(x)
print(result)
```

✓ 0.0s

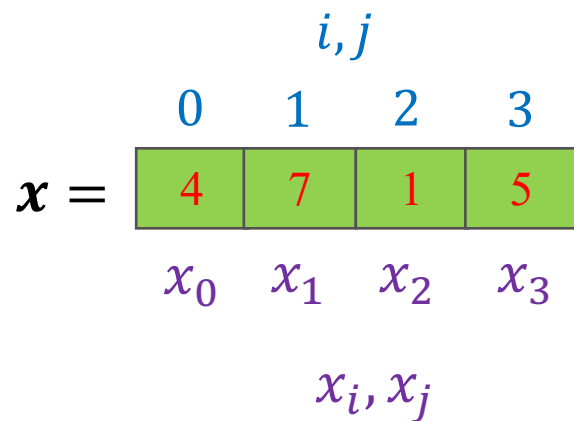
1

	0	1	2	3
$\mathbf{x} =$	4	7	1	5



Notations

❖ min and max



min (minimum)

$$\min(\mathbf{x}) = \{x_i \in \mathbf{x} | x_i \leq x_j \text{ for all } x_j \in \mathbf{x}\}$$

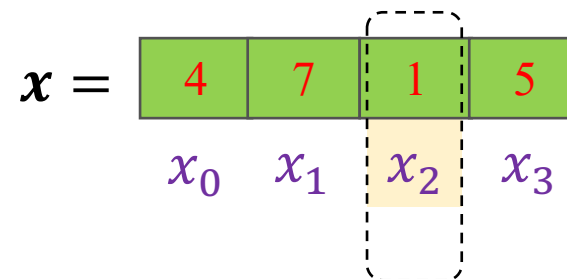
max (maximum)

$$\max(\mathbf{x}) = \{x_i \in \mathbf{x} | x_i \geq x_j \text{ for all } x_j \in \mathbf{x}\}$$

```
x = np.array([4, 7, 1, 5])
result = np.min(x)
print(result)
```

✓ 0.0s

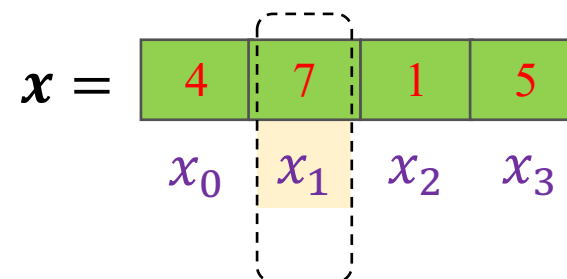
1



```
x = np.array([4, 7, 1, 5])
result = np.max(x)
print(result)
```

✓ 0.0s

7



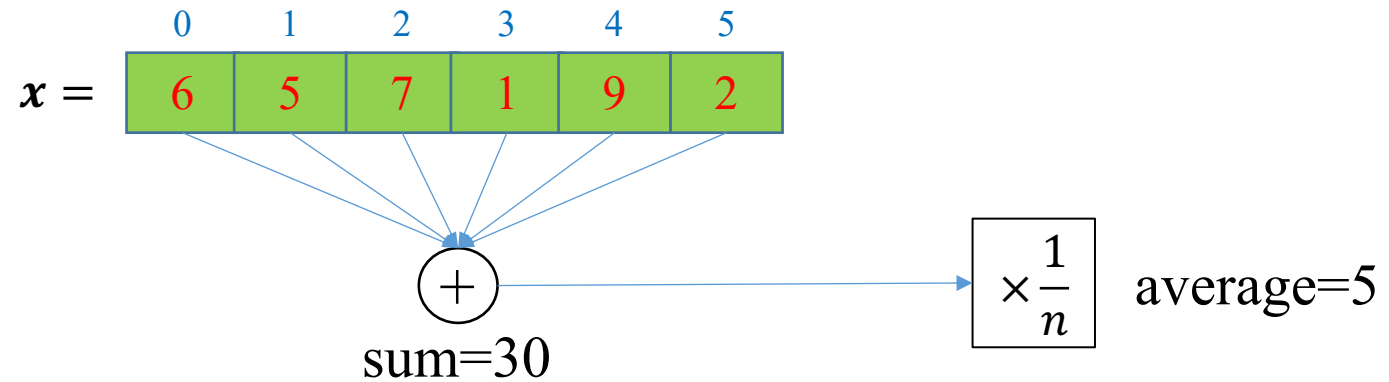
❖ Summation and average

summation

$$\text{sum}(x) = \sum_i x_i$$

Average

$$\text{average}(x) = \frac{1}{n} \sum_i x_i$$



```
x = np.array([6, 5, 7, 1, 9, 2])
result = np.sum(x)
print(result)
```

✓ 0.0s

30

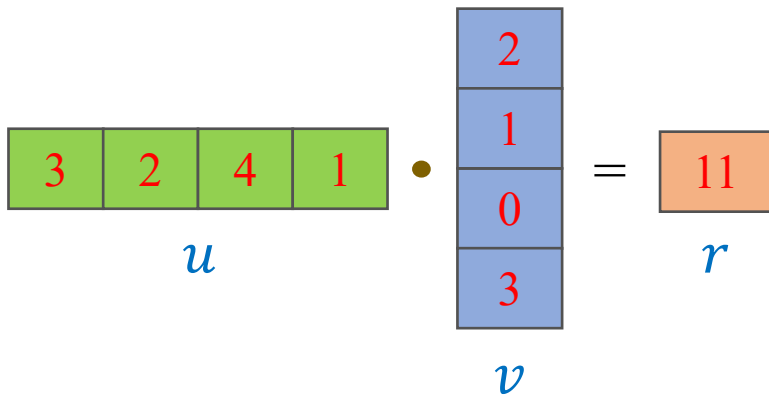
```
x = np.array([6, 5, 7, 1, 9, 2])
result = np.mean(x)
print(result)
```

✓ 0.0s

5.0

QUIZ TIME

❖ Dot product



A diagram illustrating the dot product of two 1D vectors, u and v . Vector u is represented as a horizontal row of four green boxes containing the values 3, 2, 4, and 1. Vector v is represented as a vertical column of four blue boxes containing the values 2, 1, 0, and 3. A black dot (representing the dot product) is placed between the two vectors. To the right of the dot is an equals sign, followed by a single orange box containing the value 11, which represents the scalar result r . The labels u and v are written in blue below their respective vectors, and r is written in blue below the result box.

```
import numpy as np
```

```
u = np.array([3, 2, 4, 1])
```

```
v = np.array([2, 1, 0, 3])
```

```
r = np.einsum('i,i->', u, v)  
print(r)
```

✓ 0.2s

11

 $i, i \rightarrow$

What we have:

Two inputs are two 1D vectors

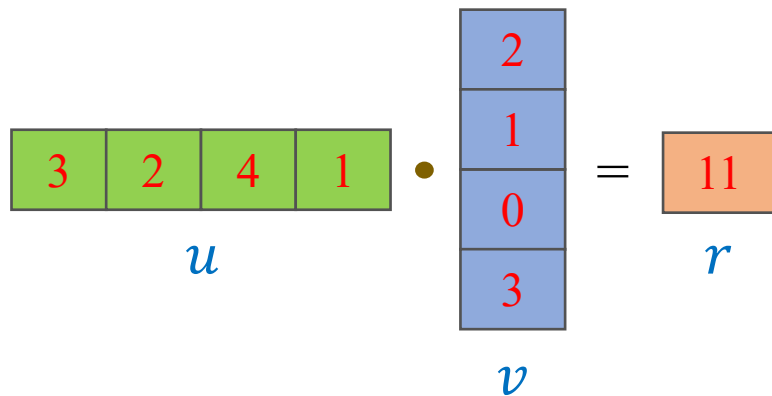
Output is a scalar

- 1) Xác định chỉ mục có bên trái nhưng không có bên phải của mũi tên.
- 2) Tính tổng trên trục chỉ mục đó

$$r = \sum_i$$

Einstein Summation

❖ Dot product



```
import numpy as np
```

```
u = np.array([3, 2, 4, 1])
```

```
v = np.array([2, 1, 0, 3])
```

```
r = np.einsum('i,i->', u, v)  
print(r)
```

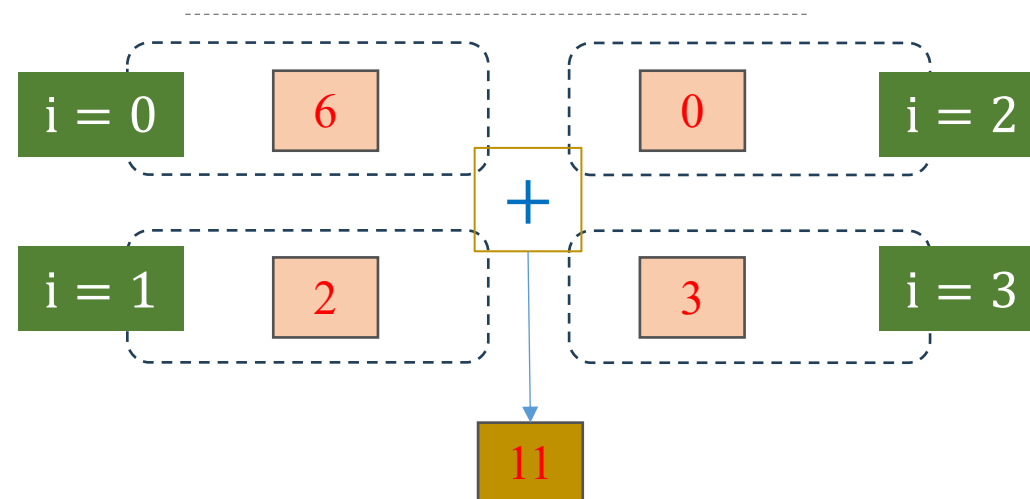
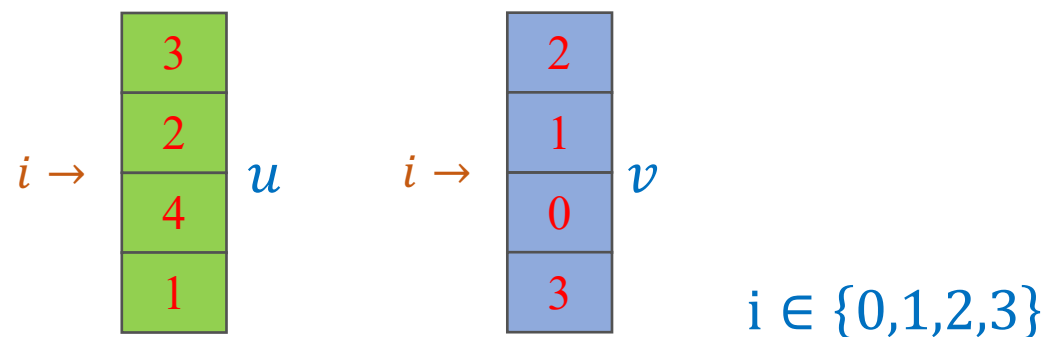
✓ 0.2s

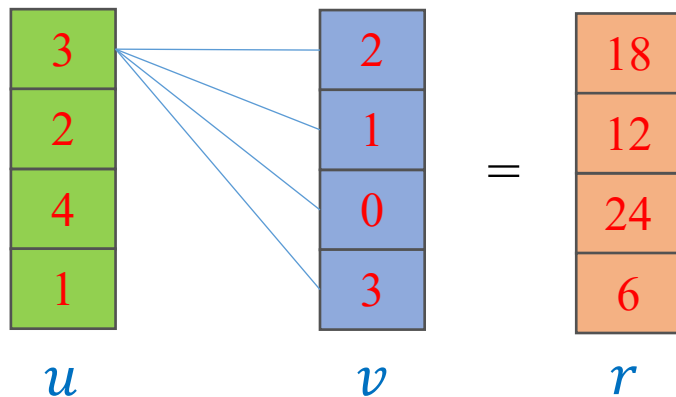
11

$i, i \rightarrow$

$$r = u_i v_i = \sum_i u_i v_i$$

Tính tổng của các phép nhân





```
import numpy as np
```

```
u = np.array([3, 2, 4, 1])
```

```
v = np.array([2, 1, 0, 3])
```

```
r = np.einsum('i,j->i', u, v)
```

```
print(r)
```

✓ 0.0s

```
[18 12 24  6]
```

$$i, j \rightarrow i$$

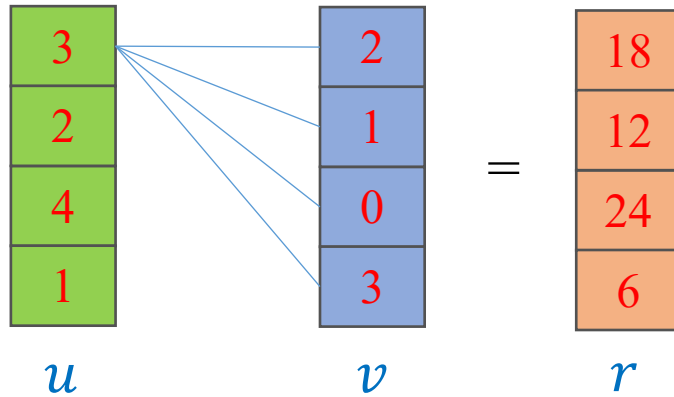
What we have:

Two inputs are two 1D vectors

Output is a vector

- 1) Xác định chỉ mục có bên trái nhưng không có bên phải của mũi tên.
- 2) Tính tổng trên trục chỉ mục đó

$$r_i = \sum_j u_i v_j$$



```
import numpy as np
```

```
u = np.array([3, 2, 4, 1])
```

```
v = np.array([2, 1, 0, 3])
```

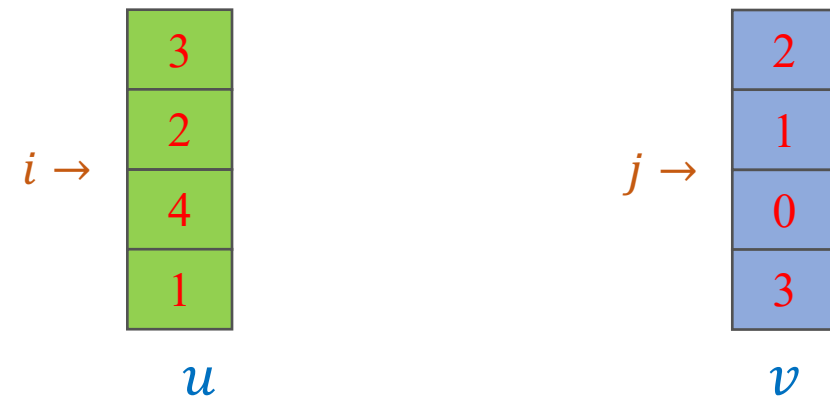
```
r = np.einsum('i,j->i', u, v)
```

```
print(r)
```

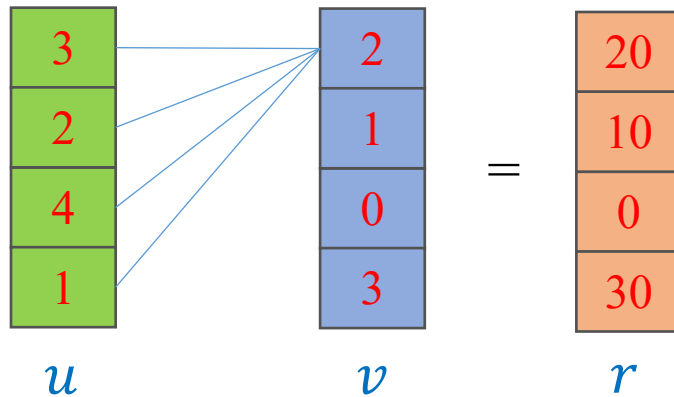
✓ 0.0s

```
[18 12 24 6]
```

$$i, j \rightarrow i$$



$$r_i = \sum_j u_i v_j = u_i v_1 + u_i v_2 + \dots$$



```
import numpy as np
```

```
u = np.array([3, 2, 4, 1])
```

```
v = np.array([2, 1, 0, 3])
```

```
r = np.einsum('i,j->j', u, v)
```

```
print(r)
```

✓ 0.0s

```
[20 10 0 30]
```

$$i, j \rightarrow j$$

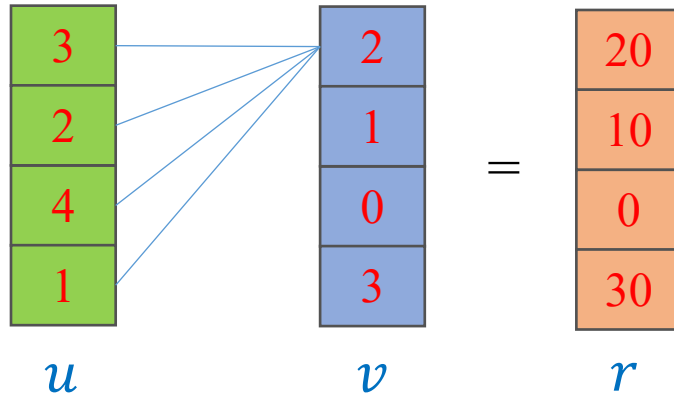
What we have:

Two inputs are two 1D vectors

Output is a vector

- 1) Xác định chỉ mục có bên trái nhưng không có bên phải của mũi tên.
- 2) Tính tổng trên trục chỉ mục đó

$$r_j = \sum_i u_i v_j$$



```
import numpy as np
```

```
u = np.array([3, 2, 4, 1])
```

```
v = np.array([2, 1, 0, 3])
```

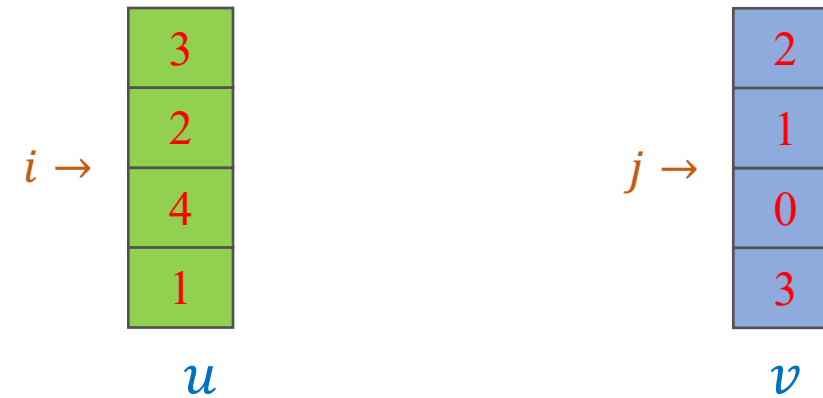
```
r = np.einsum('i,j->j', u, v)
```

```
print(r)
```

✓ 0.0s

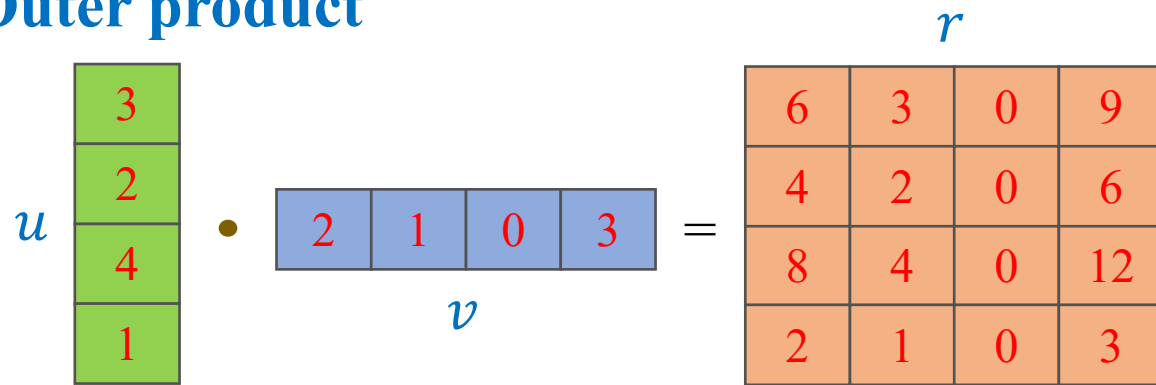
```
[20 10  0 30]
```

$$i, j \rightarrow j$$



$$r_j = \sum_i u_i v_j = u_1 v_j + u_2 v_j + \dots$$

Outer product



```
import numpy as np
```

```
u = np.array([3, 2, 4, 1])
```

```
v = np.array([2, 1, 0, 3])
```

```
r = np.einsum('i,j->ij', u, v)
```

```
print(r)
```

✓ 0.0s

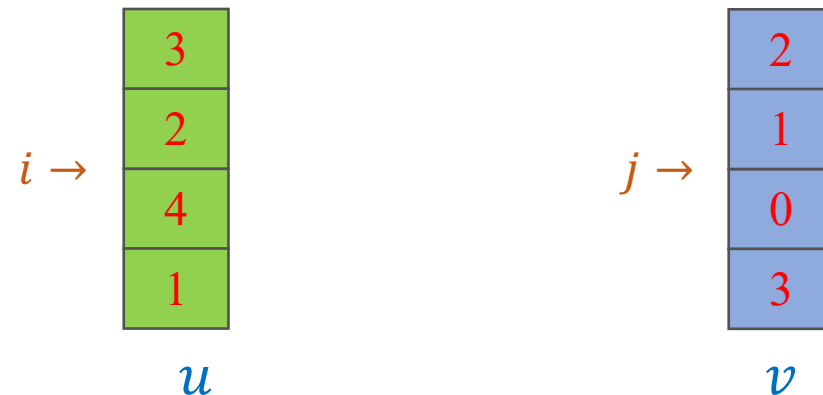
```
[[ 6  3  0  9]
 [ 4  2  0  6]
 [ 8  4  0 12]
 [ 2  1  0  3]]
```

$$i, j \rightarrow ij$$

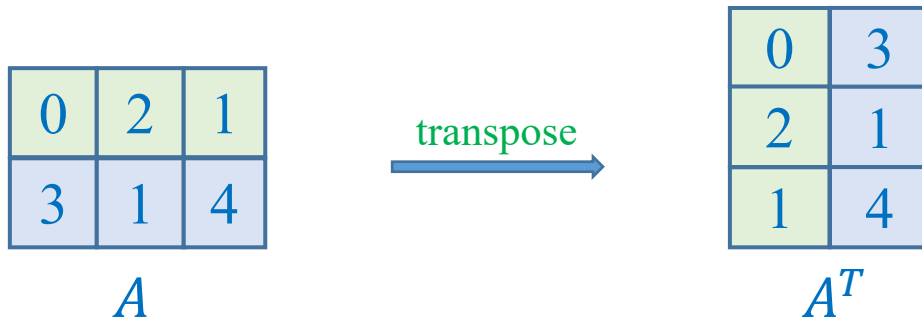
What we have:
Two inputs are two 1D vectors
Output is a matrix

Xác định chỉ mục có bên trái nhưng không có bên phải của mũi tên.

$$R_{ij} = u_i v_j$$



Transpose



```
import numpy as np

A = np.array([[0, 2, 1],
              [3, 1, 4]])
T = np.einsum('ij->ji', A)
print(T)

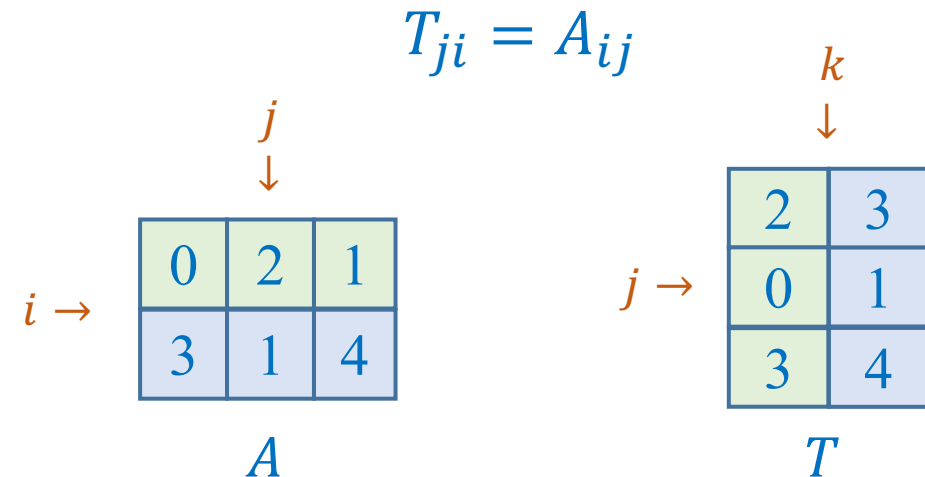
✓ 0.0s

[[0 3]
 [2 1]
 [1 4]]
```

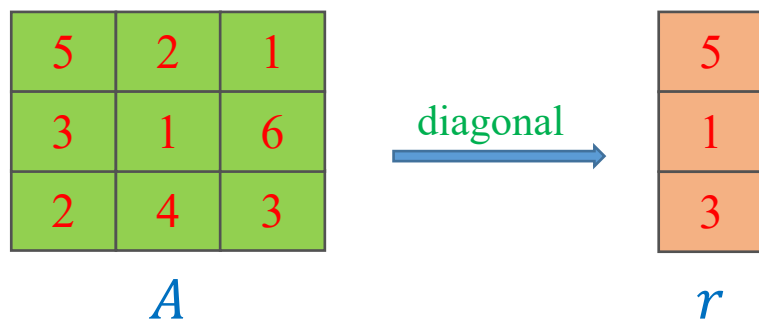
$$ij \rightarrow ji$$

What we have:
Input is a 2D matrix
Output is a 2D matrix

Xác định chỉ mục có bên trái nhưng không có bên phải của mũi tên.



Diagonal



```
import numpy as np

A = np.array([[5, 2, 1],
              [3, 1, 6],
              [2, 4, 3]])
r = np.einsum('ii->i', A)
print(r)

✓ 0.0s

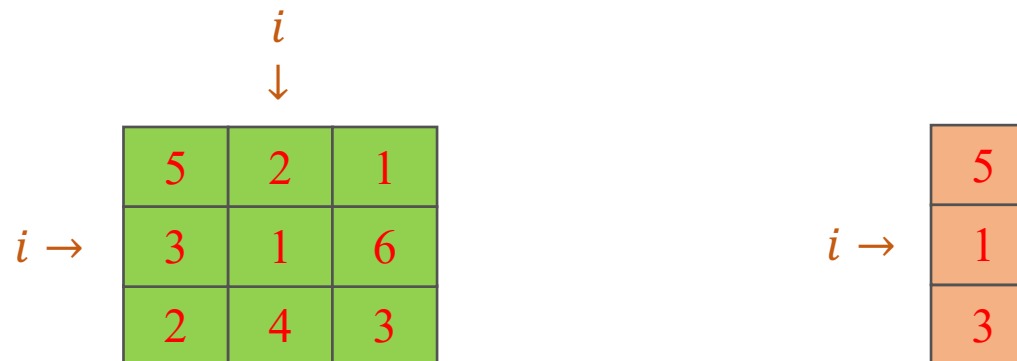
[5 1 3]
```

$$ii \rightarrow i$$

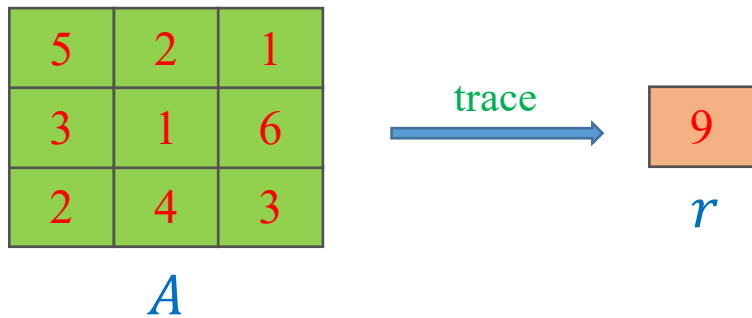
What we have:
Input is a 2D matrix
Output is a vector

Xác định chỉ mục có bên trái nhưng không có bên phải của mũi tên.

$$r_i = A_{ii}$$



Trace



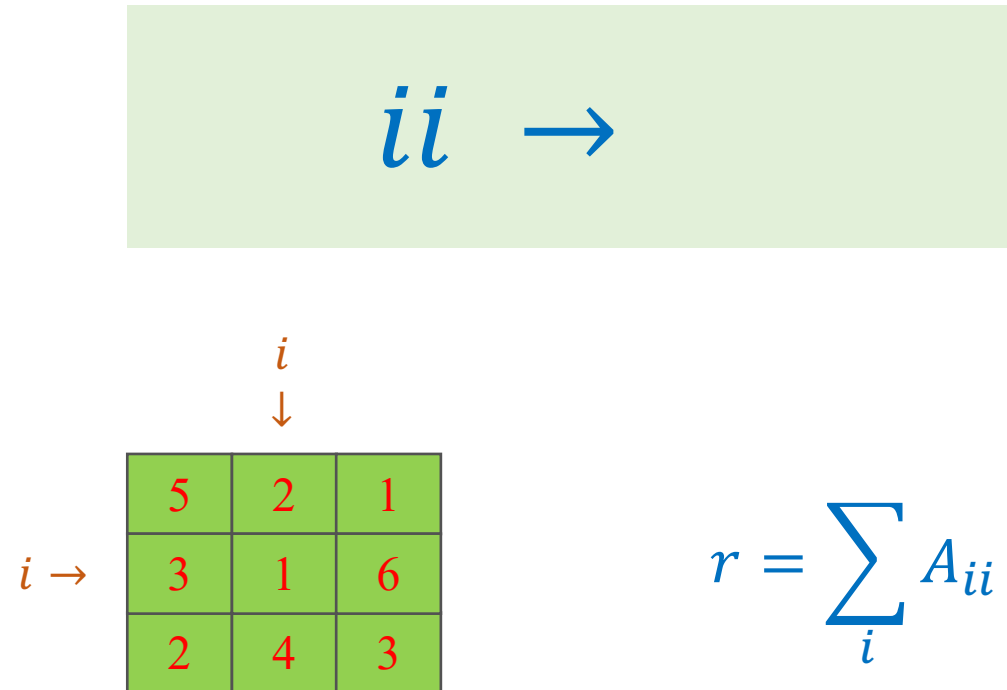
```
import numpy as np

A = np.array([[5, 2, 1],
              [3, 1, 6],
              [2, 4, 3]])

r = np.einsum('ii->', A)
print(r)

✓ 0.0s

9
```



Element-wise multiplication

3	0	1
2	2	3

 $\odot$

1	0	2
3	2	1

 $=$

3	0	2
6	4	3

$A \quad B \quad C$

```
import numpy as np
```

```
A = np.array([[3, 0, 1],  
              [2, 2, 3]])
```

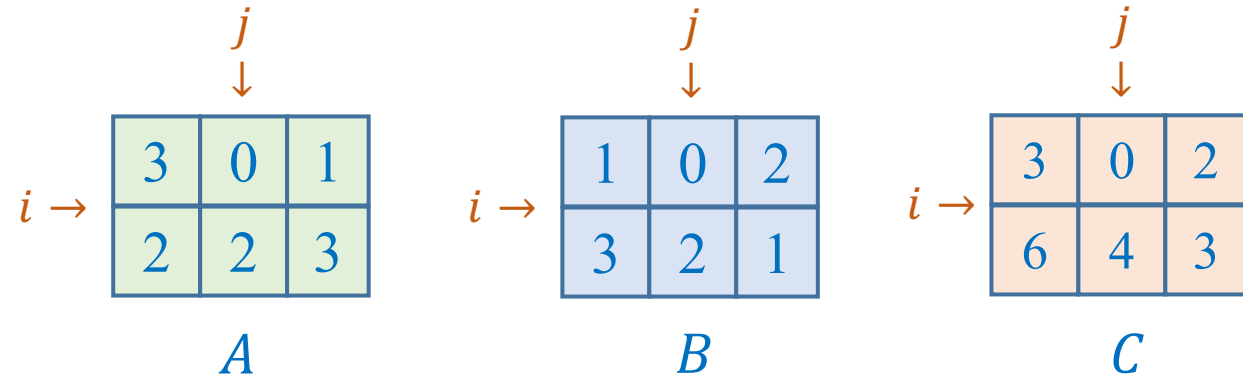
```
B = np.array([[1, 0, 2],  
              [3, 2, 1]])
```

```
r = np.einsum('ij,ij->ij', A, B)  
print(r)
```

✓ 0.0s

```
[[3 0 2]  
 [6 4 3]]
```

$$ij, ij \rightarrow ij$$



$$C_{ij} = A_{ij}B_{ij}$$

Matrix multiplication

$$A \cdot B = C$$

0	2	1
3	1	4

2	5
0	1
3	4

3	6
18	32

```
import numpy as np
```

```
A = np.array([[0, 2, 1],  
              [3, 1, 4]])
```

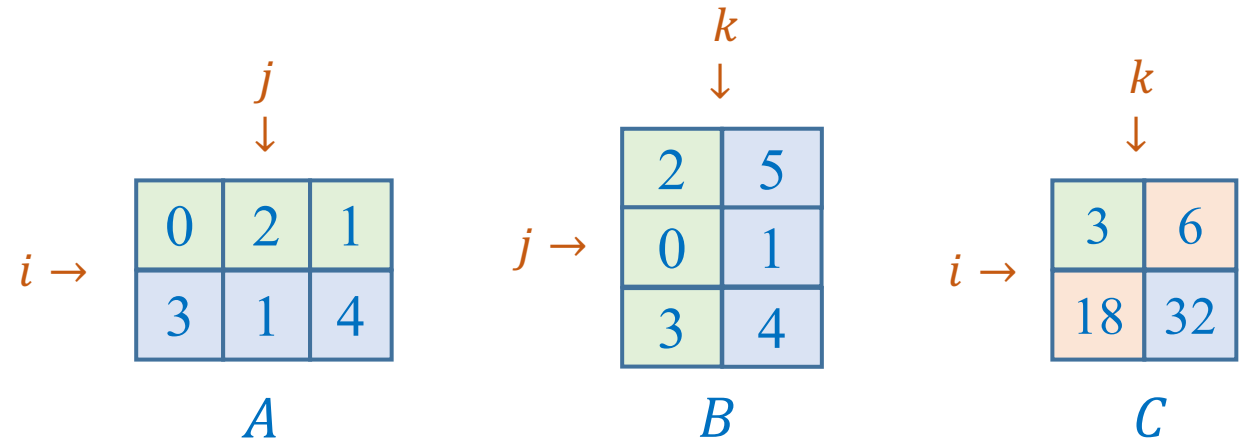
```
B = np.array([[2, 5],  
              [0, 1],  
              [3, 4]])
```

```
C = np.einsum('ij,jk->ik', A, B)  
print(C)
```

```
✓ 0.0s
```

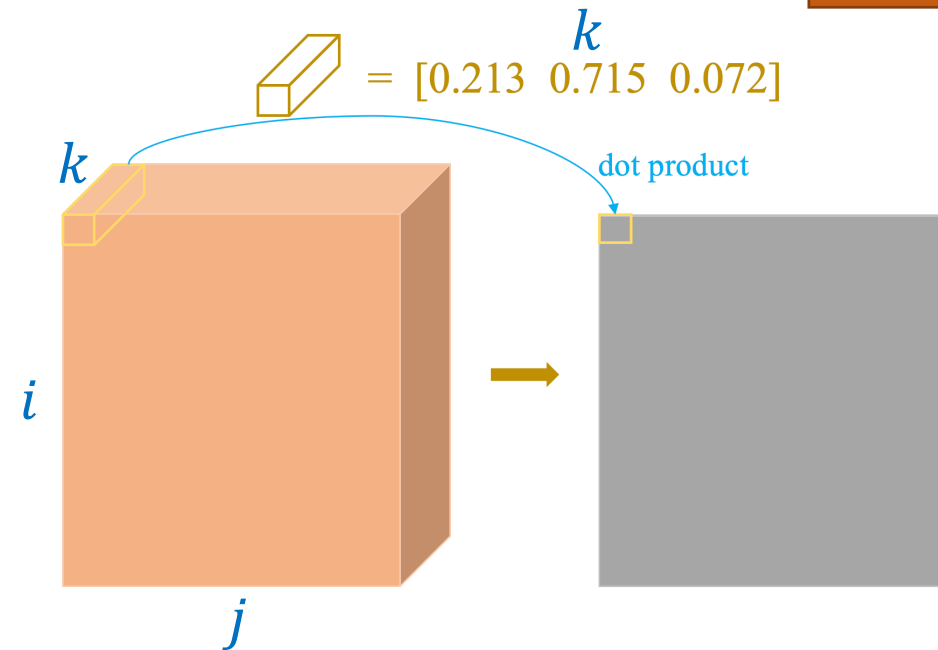
```
[[ 3  6]  
 [18 32]]
```

$$ij, jk \rightarrow ik$$



$$C_{ik} = \sum_j A_{ij} B_{jk} = A_{i1} B_{1k} + A_{i2} B_{2k} + \dots$$

Application: color2gray conversion



```
import cv2
import numpy as np

img = cv2.imread('nature.png', 1)
transform = np.array([0.072, 0.213, 0.715])

output = np.einsum('ijk,k->ij',
                   img,
                   transform)
cv2.imwrite('nature_grayscale.jpg', output)
```

